Threads and Processes

Def. 1. ***Process:*** *A program executing inside an OS (operating-system). Processes share cpu but have their own memory space.*

Def. 2. *The* ***context*** *of a process is the set of utilities it uses and or operates on. That is:*

- ***Hardware context:*** *Instruction counter, Registers, Execution State, Flags …*
- ***Memory context:*** *Virtual Memory, Heap-Space, Stack …*
- ***OS-level context:*** *Process ID, open files …*

The **Process Lifecycle** is managed by the OS and can be in one of the following states:

- **NEW / CREATED:** The process is being created and loaded from disk into memory.
- **READY:** The process is in main memory waiting to be assigned to the CPU by the scheduler.
- **RUNNING:** The process is actively executing instructions on the CPU.
- **BLOCKED / WAITING:** The process cannot continue executing and is waiting for an event (like I/O access) to complete.
- **TERMINATED:** The process has finished execution and its resources are reclaimed.

Def. 3. ***Context Switch:*** *The process of saving the context of a process and loading the context of another process.*

Def. 4. ***Process Control Block (PCB):*** *A table-entry storing relevant process information such as Program counter, Registers, Adress Space, Open files …*

Parallelism

Def. 5. ***Parallelism*** *is when multiple processes execute simultaneously on diffrent CPU-Cores.*

Def. 6. ***Concurrency*** *is when multiple processes execute interleaved on a single CPU-Core.*

Rem. 7.

$$\textit{Parallelism} \implies \textit{Concurrency}$$

$$\textit{Concurrency} \not\implies \textit{Parallelism}$$

Multithreading

Def. 8. *A thread is an independent sequence of exection running in the same OS-process.*

Rem. 9. *While processes can only communicate through main-memory and have diffrent memory spaces, threads share the same adress space. This leads to faster and more efficient context-switches as adressspace and PCB loading is not necessary.*

There are diffrent levels of threads:

- **User-Level Threads:** Threads managed by the application using a thread library, ex. Java-Threads managed by JVM
- **Kernel-Level Threads:** Threads managed by to OS, which can schedule them on diffrent cores.
- **CPU-Level Threads:** Threads managed by the CPU, which can schedule them on the physical cores.

There are two ways to create a thread in java.

Code Example 10

```
class MyThread extends Thread { ...
    public void run() {
        ...
    }
}
MyThread thread = new MyThread();
thread.start(); // calls MyThread.run()
```

or using a runnable, which is recomended to do:

Code Example 11

```
class MyRunnable implements Runnable { ...
    public void run() {
        ...
    }
}
MyRunnable runnable = new MyRunnable();
Thread thread = new Thread(runnable);
thread.start(); // calls MyRunnable.run()
```

In its lifecylce a thread can have the follwoing states:

- **NEW:** The thread has been created but not yet started.
- **RUNNABLE:** The thread is ready to run and waiting for the OS to schedule it.
- **RUNNING:** The thread is currently executing.
- **BLOCKED:** The thread is waiting for access to I/O devices
- **WAITING:** The thread is waiting to be notified
- **TERMINATED:** The thread has finished its execution

Rem. 12. *Every Thread has an* ***ID, Name, Priority and State*** *which might be usfull for certain tasks.*

To wait for a thread to finish, we can use the .join() method on the thread we want to wait on. This lets the main-thread sleep until the joined thread terminates and wakes it up again.

Code Example 13

```
Thread t = new MyThread();
t.start(); //start t
t.join(); //sleep until t is finished
continueWork(); // this is only called after t
        finishes
```

Rem. 14. *.join() creates overhead, for small thread-exection-time this is ineffiecient.*

Imp. 15. *If a worker-thread throws an exception the main-thread* ***will be woken up, but it will not know of the error****. We have to set an* ***UncaughtExeptionHandler using a try-catch block*** *when joining threads. For threads that run indefinetly we use a* ***timer task*** *that interupts threads.*

Code Example 16

```
MyTimerThread extends Thread {
    run(Thread thread, int time) {
        timer.sleep(time);
        thread.interupt();
    }
}

Thread t = new MyThread();
Thread timer = new MyTimerThread();
t.start();
timer.start(t, 1000);
try {
    t.join();
} catch (UncaughtExeptionHandler) {
    System.out.println("Error in thread t");
}
continueWork();
```

Imp. 17. ***Thread scheduling is not deterministic****. Don't rely on the execution order (interleaving).*

Synchronization

Def. 18. ***Bad Interleaving:*** *Interleaving of operations from multiple threads that leads to incorrect program state.* ***Critical Section:*** *Codeblocks that might lead to bad interleavings.*

Def. 19. ***Thread Safety:*** *Nothing bad happeing in all possible interleavings.*

Def. 20. ***Thread Liveness:*** *Something good happening eventually.*

Locks and Synchronized

Every Java-Object has an **intrinsic lock** or lock monitor. These locks are **unique** and can be optaind by threads which allows us to lock a certain codeblock.

Code Example 21

```
synchronized(lockObject) {
    i++;
}
```

Rem. 22. *Synchronized can also be used as a keyword on a method to sync. the whole method.*

Rem. 23. *By default synchronized uses "this" as the lock-object and unlocks when an exception occurs.*

Rem. 24. *Java-Locks are reentrant meaning they can be locked multiple times by the same Thread recursively (It will then have to release the look as many times as it aquired it).*

Atomic integers have builtin synchronization, for example for incrementing/decrementing.

Code Example 25

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
```

Wait, notify, notifyAll

We can use wait() on a lock-object to tell the current thread to sleep until further notice if the lock is not available. This allows use to save ressources.

Code Example 26

```
synchronized(lockObject) {
    while(!condition) {
        lockObject.wait();
    }
    // do work
}
```

Imp. 27. *Always wait inside a while loop, as this ensures checking the condition after beeing notified.*

To wake one arbitrary or all waiting threads we can use notify() or notifyAll() on the lock-object.

Code Example 28

```
synchronized(lockObject) {
    while (!condition) {
        lockObject.wait();
    }
    //do work
    notifyAll();
}
```

Lock API

Instead of using built-in synchronized blocks we can also use locks from the java.util.concurrent.locks package.

Code Example 29

```
private Lock lock = new ReentrantLock();
...
void doWork(int x) {
    lock.lock();
    try {
        ...
    } finally {
        lock.unlock();
    }
}
```

Imp. 30. *Always use a* ***try-finally block to ensure the lock is released*** *even if an error occures.*

Rem. 31. *Use lock.tryLock(100, TimeUnit.MILLISECONDS) to accuire the lock only for a given amount of time, avoiding deadlocks.*

Pros and Cons of synchronized vs Lock API:

Aspect	synchronized	Lock API
Handling	Auto (JVM)	Manual (lock/unlock)
Release	Automatic	Manual (finally!)
Scope	Block/Method	Flexible/Multi-method
Waiting	Infinite block	tryLock() avoids block
Interrupt	No	lockInterruptibly() ok
Flex.	Low	High
Comp.	Simple	Complex/Error-prone
Risk	Lower	Higher (forget unlock)
Usage	Simple sync	Advanced control

Optimization

Thm. 32. ***Moore's Law****: Number of transistors on a microchip doubles every two years. Hitting the* ***Power-Consumption*** *(cooling limits) and* ***Memory*** *walls necessitated parallelism.*

Caching

Faster CPU's does not mean faster execution. Writing and loading data forms a bottleneck.

Def. 33. ***Cache*** *is a fast memory that preloads and stores frequently used data form the memory to reduce latency.*

The usage of cache reduces stalls (times in which the CPU can not execute as the data is not yet present).

Rem. 34. *Programmers improve cache usage by organizing data to increase* ***locality****, enabling efficient prefetching.*

Vectorization

Def. 35. ***SIMD*** *(Single Instruction Multiple Data) same instruction is applied to multiple data points simultaneously by using multiple ALUs inside the CPU.*

Superscalar and Speculative Execution (ILP)

Def. 36. ***Superscalar execution*** *refers to out of order execution of independent instructions in parallel using multiple ALUs.*

Def. 37. ***Speculative execution*** *refers to the execution of instructions before the actual need for them is known.*

Pipelining

Pipelining increases throughput by overlapping the execution of multiple instructions in diffrent stages.

Def. 38. ***Throughput*** *is the amout of work that can be done by a system in a given period of time.*

$$tp = \frac{1}{max(compTime(stages))}$$

Def. 39. ***Latency*** *is the time needed to perform a given computation.*

$$L = numStages * max(compTime(stages))$$

Def. 40. *A pipeline is* ***balanced*** *if all stages take the same amount of time.*

Imp. 41. *Balancing a pipeline can increase throughput.*

Parallel Performance

Def. 42. ***Scalability*** *describes how performance improves with additional resources (cores, memory, etc.).*

Def. 43. ***Speedup*** *is the ratio of the execution time of the sequential algorithm to the execution time of the parallel algorithm.*

$$S_p = \frac{T_1}{T_p}$$

Def. 44. ***Efficiency*** *is the ratio of speedup to the number of processors.*

$$E_p = \frac{S_p}{p}$$

Rem. 45. *Speedup depends on the definition of T_1 (execution time of the optimized sequential algorithm vs. the unoptimized one).*

Thm. 46. *Amdahl's Law sets a limit on speedup, as the sequential part of the algorithm can not be parallelized.*

$$S(p) = \frac{1}{1 - f + \frac{f}{p}}$$

Thm. 47. *Gustafson's Law shows that we can perform more operations in the same execution time.*

$$S_p = f + P(1 - f)$$

Divide and Conquer

Def. 48. ***Divide and Conquer*** *refers to the strategy of splitting a problem into smaller subproblems, solving them recursively, and combining the results.*

Problems of DC with manual fork/join:

- Poor reusability of code
- Commit to a number of threads a priori and can not dynamically adjust.
- Subproblems are not automatically assigned to a thread.

Rem. 49. *Some manual fork/join fixes are: sequential-cutoffs or running one subproblem on the thread itself, instad of creating two new threads.*

Executor Service

The Executor Service framework provides the way to schedule and run tasks on a thread-pool automatically.

Def. 50. ***Runnable****: Task without return value. (Passed to* `execute()` *or* `submit()`*).* ***Callable****: Task returning* `Future<T>`*. (Passed to* `submit()`*).*

Rem. 51. ***Pool Types:*** `newFixedThreadPool(n)` *(keeps n threads),* `newCachedThreadPool()` *(scales dynamically).*

Pros and Cons of Executor service:

- Not suited for: Recursive subproblems that depend on each other and have a deep fork-tree. (run out of threads)
- Suited for: flat structures that can run independently in parallel.

Fork-Join

Manages a pool of threads to solve recursive tasks using a work-stealing strategy, where threads have local task-queues which they work on and if empty steal from other threads.

Def. 52. ***RecursiveTask****: Returns a result (like Callable).* ***RecursiveAction****: Does not return a result (like Runnable).* ***Key Methods:*** `fork()` *(async push),* `join()` *(wait, but steals work while waiting),* `compute()` *(main logic).*

Code Example 53

```
class MyTask extends RecursiveTask<Integer> {
    int start, end;
    MyTask(int start, int end) { this.start =
            start; this.end = end; }

    protected Integer compute() {
        if (end - start <= SEQUENTIAL_CUTOFF) {
            return baseCase();
        } else {
            int mid = (start + end) / 2;
            MyTask left = new MyTask(start, mid);
            MyTask right = new MyTask(mid, end);

            left.fork(); // Async execute left
                    half
            int rightRes = right.compute(); //
                    Optimize: compute right half
                    now
            int leftRes = left.join();      //
                    Wait for left half

            return leftRes + rightRes;
        }
    }
}
```

Imp. 54. *Make sure that every task has enough work to do (aprox. 100-10'000 ops.) (else the overhead will be larger than the parallel speedup gain)*

Task graphs

The execution of parallel programs using Fork-Join can be represented as a Task-Graph (DAG).

Def. 55. ***Span*** *T_∞: execution time with unlimited hardware (Sum of longest dependent path in DAG). Max speedup is limited by T_1/T_∞ (Amdahl's Law).*

Parallel Algorithms

Parallel Patterns

Def. 56. *A* ***reduction*** *produces a single answer from a collection via an associative operator.*

Ex. 57. *Max, Min, Sum, Product of an array.*

Def. 58. *A* ***map*** *applies a function to each elements of a collection producing an output of the same size.*

Ex. 59. *Vector addition: $v_i = u_i + w_i \forall i \in [1, n]$*

Rem. 60. *Maps and Reductions have an optimal span of $O(\log n)$.*

Def. 61. *A* ***stencil*** *extends the map pattern by using the element itself and its neighbours.*

Ex. 62. *A smoothing kernel (ex. 3x3) that calculates the average value is sled over an image to smooth/blur the image.*

Parallel Algorithms

Prefix-Sum

Prefix-Sum: Sum of first n inputs.

Algorithm 63

```
upTask(); //divide problem and build tree with sums of subproblems
downTask(); //fill in the correction values (left_sum + parent_correction)
```

Thm. 64. *The span of the prefix sum optimization is $O(\log n)$, therefore parallelism of $\frac{n}{\log n}$.*

Pack

Pack pack all elements from an array that satisfy a certain condition into a seperate output. (by using prefix sum)

Algorithm 65

```
applyMap(); //applies a map to the input
//results in a binary vector [1, 0, 1, 1, 0, ...]\\
parallelPrefixSum(); //calculates the prefix sum of the binary vector\\
//results in [1, 1, 2, 3, 3, ...]\\
applyPackMap(); //applies another map to the result of the prefix sum\\
//results in [0, 2, 3, ...] (indices of elements to write to output)
```

Quicksort

We can reduce the **limiting factor of the problem division** from a span of $O(n)$ to a span of $O(\log n)$ using the pack-pattern.

Algorithm 66

```
applyMaps(); //apply maps for greater and smaller than pivot
parallelPrefixSum(); //count how many smallerthan/greatherthan elements come bofore
applyMap(); //index = numLessThan || numTotalLessThan + numGreaterThan
```

We can do this by using two pack-patterns to partition the data. This leads to a partitioning of $O(\log n)$ levels times $O(\log n)$ for prefixSum hence $O(\log^2 n)$ total span.

This means a parallelism of $\frac{n \log n}{\log^2 n} = \frac{n}{\log n}$.

Shared Resources

Race Conditions

Def. 67. ***Race condition:*** *correctness depends on specific interleaving.*

Def. 68. ***Low-level race condition:*** *insufficient synchronization of memory access (Data Races).* ***High-level race condition:*** *wrong decomposition/algorithm (bad interleavings despite mutual exclusion).*

Generally we want one of three things when it comes to shared ressources:

- **Thread local (Isolated Mutability):** We dont use the ressource of other threads.
- **Read only (Immutable):** We only read and do not change the ressource
- **Mutual exclusion (Synchronization):** We use the ressource of other threads but we ensure that only one thread accesses it at a time.

Guidelines for Concurrent programming

Imp. 69. ***Thread-Locality:*** *Only share ressources if needed.*

Imp. 70. ***Immutability:*** *Use immutable data structures whenever possible.*

Imp. 71. ***Consitent locking:*** *Use locks to ensure mutual exclusion.*

Def. 72. ***Lock granularity*** *refers to the amount of code that is proteted by a lock in ways:*

- *Size of synchronized blocks*
- *Number of objects a lock is used for*

Thm. 73. ***Coarse-grained locking*** *is easier to implement but leads to less concurrency.* ***Fine-grained locking*** *is harder to implement without creating bugs but leads to more concurrency.*

Imp. 74. *Start with coarse-grained (simpler) and move to fine- grained (performance) only if contention on the coarser locks becomes an issue.*

Def. 75. ***Atomicity:*** *An operation is atomic if it is performed as a single, indivisible unit.*

Imp. 76. *Do not do expensive computations or I/O in critical sections, but also don't introduce race conditions*

Memory Models

Def. 77. ***Memory Reordering:*** *Compilers/hardware may reorder memory operations as long as sequential semantics hold (optimizes registries, caching, dead code). May break concurrency (e.g., hoisted checks $\implies$ infinite loops). L1 caches are private, writes may not immediately publish.*

Rem. 78. `volatile`**:** *Prevents local caching/reordering skips; ensures immediate visibility across threads. Memory barriers strictly enforce ordering.*

Java Memory Model (JMM)

Contract providing minimal memory operation guarantees.

Def. 79.

- ***Program Order (PO):*** *Total intra-thread execution trace order (no global sync).*
- ***Sync Actions (SA):*** *E.g., Read/Write* `volatile`*, lock acquire/release.*
- ***Sync Order (SO):*** *Total global order of all SA across all threads.*
- ***Happens-Before (HB):*** *Transitive closure of PO & SA (Synchronizes-With). $A \xrightarrow{HB} B \implies B$ sees results of A.*

Rem. 80. ***HB-Consistency & Data Races:*** *JMM permits data races (unsynchronized concurrent access where ≥ 1 is a write). Unsynchronized reads may observe stale values or non-HB ordered writes.*

Behind Locks

Properties for correct Lock (Mutex) execution:

- **Mutual Exclusion:** ≤ 1 process in critical section (CS).
- **Deadlock Free:** 1+ processes trying $\implies$ 1 eventually succeeds.
- **Starvation Free:** Every trying process eventually succeeds.

Def. 81. ***State Space Diagram:*** *Tracks every concurrent configuration. States are tuples $[p, q, want_p, want_q]$. Used to prove lock correctness (find states with No Mutex, Deadlock, Starvation).*

Def. 82. ***Atomic register:*** *Operations occur at a single instant. Event $A \prec B$ (precedes) if execution intervals don't overlap.*

Mutual Exclusion Algorithms

Peterson's Algorithm (2 Processes): Combines flags and a *victim* tie-breaker. Guarantees Mutex, Deadlock Freedom, and Starvation Freedom. **Code Example 83**

```
wantp = true;
victim = 1; // "Give way" to other thread
while (wantq && victim == 1); // spin
/* CS */ wantp = false;
```

Rem. 84. ***N-Processes: Filter Lock:*** *$N-1$ levels, uses Peterson's at each level to block 1 victim.* ***Bakery Algorithm:*** *Threads take ticket > all outstanding. Wait for lowest number.*

Hardware Support for Parallelism

Provides $O(1)$ space locks via **atomic instructions** (Java: `AtomicInteger`, etc).

Def. 85.

- ***Compare and Swap (CAS):*** *Atomically update memory if equal to expected value.*
- ***Test and Set (TAS):*** *Atomically read memory and set to 1.*
- ***LL/SC:*** *LL sets "link". SC writes only if link still valid (no other writes).*

Atomic Locks

TAS Lock: `while(state.getAndSet(true));` Causes high cache coherence traffic (continuous atomic updates).

TATAS Lock: Spin on *read* before trying *write*. **Code Example 86**

```
public void lock() {
    do { while (state.get()); }
    while (state.getAndSet(true));
}
```

Rem. 87. ***Backoff Strategy:*** *E.g., exponential wait after failed acquire, significantly reduces cache traffic and contention.*